Campus Food Bank Booking System

What we've decided so far, and the ifs and buts

Prakash Periyasamy · 27 October 2026 · Status: planning, no code yet

The idea

Memorial University's Campus Food Bank in St. John's gives students free food. This project is a booking website for it. Students pick a pickup time and choose items from what is in stock. Volunteers get a list of orders to pack. The system enforces the food bank's rules, such as one pickup every 28 days, and it is private by design: it never stores names, emails or phone numbers.

> **What it is and isn't.** It's an independent project I'm building in my free time. The first version is a prototype with made-up items and sample rules. It will only use fake data until the Campus Food Bank (and MUNSU) agree to anything real.

Who uses it

Student	No account. Books with their MUN ID number and gets a booking code. Needs both to view, change or cancel.
Volunteer	Logs in. Sees the packing list for their shift, swaps in substitutes, records walk-ins and extras, and marks orders packed, picked up or no-show.
Coordinator	Logs in. Manages items, stock, settings, closed days and volunteer accounts. Sees totals only, never individual students.

How a booking works

1	**Book**	The student enters their MUN ID number and picks a 15-minute slot up to 7 days ahead.
2	**Choose food**	They choose items (up to 13 for one person) and rank up to 3 substitutes in case something runs out.
3	**Confirm**	The items are set aside for them. They get a booking code and can download a calendar file as a reminder.
4	**Pack**	Volunteers pack the order. If an item is gone, they use the student's next substitute.
5	**Pick up**	The volunteer types the number from the student's MUN card. The system checks it matches the booking. Extras such as bread or eggs are asked for at the counter.

What we decided, and why

Each decision is logged with its reason in DECISIONS.md in the project folder.

Technology

FastAPI, PostgreSQL and React	PostgreSQL because the booking rules need real transactions and row locks (so two people can't grab the last slot at the same moment). SQLite can't lock individual rows.
Prototype first, with sample rules	The real items and rules aren't known yet. Every number that might change (the 28 days, the 13 items, slot sizes) is a setting, so switching to the real rules means changing settings, not code.
The database enforces the key rules	Slot capacity, the daily limit, stock never going below zero, and one upcoming booking per person are enforced by the database itself, so they hold even if the app code has a bug.
Always lock the day before the slot	A booking updates two counts. Locking them in the same order every time stops two bookings from waiting on each other forever (a deadlock).

Privacy

Identity is the MUN ID number, not the MCP (health card) number	International and out-of-province students don't have an MCP card, and a health card number is too sensitive to hold. Accepting only one kind of ID stops one person having two identities.
Store a scrambled version of the ID, never the number	Student numbers are only 9 digits, so a plain hash could be reversed by trying all billion numbers. A keyed hash (HMAC) can't be reversed without a secret key kept outside the database.
No names, emails or phone numbers	Not even in logs. Reminders come from a calendar file the student downloads instead.
Volunteers never see the ID	They see a booking code and the items only.

Fairness rules

Walk-ins go into the same system	Otherwise someone could walk in one week and book online the next, getting around the 28-day rule.
Up to 13 items, not exactly 13	Students shouldn't be forced to take food they won't use.
No-shows don't count toward the 28 days	People miss pickups for many reasons, and a penalty hurts the people who need food most. It's a setting, so the food bank can change it.
One of each item, whatever the variety	One canned chicken, even if there are several kinds and plenty in stock. This follows the food bank's rule.
Extras: one of each, chosen at pickup	Extras (eggs, bread, chips) change too often to book online, but they still need a limit so they're shared fairly.

Substitutes (when something runs out)

Students rank up to 3 substitutes when booking	Volunteers swap them in while packing, so nobody has to decide at the counter. Three, so a student still gets a full order if a few items are missing.
Substitutes aren't set aside	Setting them aside would hold back 3 extra items per student that usually aren't needed, and other students couldn't book them.
Never swap in something the student didn't list	A student who picked sunflower oil may be avoiding peanut oil because of an allergy. Only the student knows, so only their list is used.
Fill gaps from the list in rank order	Any substitute can fill any gap, as long as it's in stock and doesn't give a second of the same item. Simple for volunteers and easy to test.

The ifs and buts

Every choice above has a cost. These are the trade-offs and weak spots we know about.

If we do this…	…but
Substitutes aren't set aside.	A substitute may also be gone by packing time. If all of them run out, the order goes out smaller.
Booking online doesn't prove you're a student.	Anyone can type a number. That's why the volunteer checks the MUN card at pickup.
Household size is self-reported.	Someone could claim a bigger household to get more items. We don't collect family details, so it can't be checked.
Two adults in the same household can each book with their own MUN ID.	The system can't detect this without collecting addresses, which would break the privacy rules. Accepted as a known gap.
No-shows aren't penalised.	A missed booking holds its slot and items until 30 minutes after the slot ends, then they go back. If this gets abused, the food bank can switch the penalty on.
No email or text reminders.	Students rely on the calendar file. The trade is fewer reminders for storing no contact details.
All the numbers are samples.	13/18/23 items, 4 people per slot, 40 per day, opening hours: all placeholders until the food bank confirms the real rules.
The ID number works like a weak password.	See *Still to decide* below. This is the biggest open problem.

Still to decide

1. The ID number works like a weak password.
Anyone who knows a student's number can book in their name. Because each person can only have one upcoming booking, that blocks the real student, and they can't cancel it without the booking code. Someone could also fill slots using made-up numbers.

Proposed fix: in the prototype, a volunteer can cancel a booking when the real student shows their MUN card, and there's a limit on how many bookings one device can try. If it ever runs on MUN's site, students would log in with their MUN account instead.

2. Show substitutions on the booking page?
For example: *Sunflower oil → rice*, so the student knows what changed before they arrive.

3. AI meal suggestions.
Suggest meals from the items a student picked. Deferred: decide later.

Questions for the Campus Food Bank

- How do household sizes work? Do families get more items?
- How is the 4-week rule checked today?
- How do you handle two people from the same household?
- Can someone pick up for another person?
- How are students from other colleges served?

Sample settings

The coordinator can change these without touching the code. These values are placeholders.

Setting	Sample value
Pickup gap	28 days
Booking window	7 days ahead
Booking and cancel cutoff	3 hours before the slot
Opening hours	Sun 3–5 pm; Mon and Thu 3:30–5:30 pm and 7–8 pm
Slot length / capacity	15 minutes / 4 people
Daily limit	40 people, with 10 places kept for walk-ins
Substitutes	Up to 3 per booking
No-show time	30 minutes after the slot ends
Booking details deleted after	90 days (only totals are kept)

Household	Item limit	Max of each item
1 person	13	1
2–3 people	18	1
4 or more	23	1

Not in the first version

- Reminders by email or text
- Picking up for someone else
- Students from other colleges, or other campuses
- Other languages
- AI meal suggestions (to decide later)

What happens next

1. Settle open points 1 and 2 above (the weak-password fix, and showing substitutions).
2. Design the database tables: map every rule to a table, a database rule, or app code.
3. Build the prototype with fake data and demo it.
4. Talk to the Campus Food Bank to get the real rules and items.